Technical report — Alpenstroke

An AI swim training log on Apertus: write a workout the way you would text a coach, in any language, and get structured sets, training-load metrics, an analysis of the last six weeks and a plan for the next seven days.

- **Track:** Track 2B — Alpenstroke
- **Event:** Online
- **Team:** Alpenstroke — Nikita Kolesnikov (solo)
- **Demo video (2 min):** https://drive.google.com/file/d/1eK1xKbC7viZwHzX7OZr3-u9bdMsOG8f-/view · live instance: https://alpenstroke.nkolesnikov.dev
- **Code:** https://github.com/nikitojik/Alpenstroke (this directory: `track_2b/`)

1. Summary

Competitive swimmers log a lot of training, usually in a notebook or a rigid app where every set is typed into a form. Neither tells the swimmer what matters before a meet: *am I overdoing it, and is something going wrong?* Alpenstroke lets a swimmer describe a session in plain words in English, German, French, Italian or Russian. Apertus turns the text into sets; plain code computes the load (session RPE, weekly volume, acute:chronic workload ratio); Apertus then explains the numbers and plans the next week.

The design principle is **code does the math, the model does the language**. Every model answer is validated against a schema and then checked and corrected by deterministic code: the pool length is detected by pattern, each set must quote the words it came from, a quoted stroke name overrides the model, invented body parts trigger a retry, the weekly volume target is computed and enforced by code. On 27 hand-written multilingual logs this took fully correct parsing with Apertus 70B from **54% (prompt only) to 95%**; on 6 blind test cases written before the last round of changes it reached **100%** in the latest run. Workout analysis is correct in **100%** of runs for both Apertus 70B and Apertus v1.5 8B. The whole system runs **air-gapped** with Apertus v1.5 8B in llama.cpp on a Docker network without a route to the internet, which a built-in check proves.

2. Architecture

```
browser ──► FastAPI (Jinja2 + HTMX, no CDNs) ──► PostgreSQL 16
                 │
                 ├─ parse:    free text ──► Apertus ──► schema ──► code checks (pool, strokes,
                 │                                                  rest vs send-off, numbers)
                 │            ──► draft ──► swimmer confirms ──► saved
                 ├─ metrics:  plain code (session RPE × minutes, weekly volume, ACWR 7/28 days)
                 ├─ analyze:  metrics + last 14 sessions + notes ──► Apertus ──► schema
                 │            ──► grounding check (body parts must appear in the swimmer's notes)
                 └─ plan:     volume target (code) + latest analysis ──► Apertus ──► schema
                              ──► rule checks (≤2 hard days in a row, day ≥ its main set,
                                  caution if a symptom) ──► dates, race day, scaling to target (code)
```

All model calls go through one function (`src/app/services/llm.py: chat_json`). It adds the JSON Schema of the expected answer to the system prompt, validates the reply with Pydantic, runs a task-specific check, and on any problem sends **one retry** that lists the exact problems (for example "*Set 2: distance 200 does not appear in its source '300 stile libero defaticamento'*"). The athlete's name is never sent to the model.

Code checks per task (`src/app/services/set_rules.py`, `grounding.py`, `planner.py`):

Task	What code decides or verifies
Parse	pool length from the text in five languages; stroke from the quoted source (one shared glossary for prompt and code); rest vs send-off; every distance and rep count must appear in its quote; no two sets may quote the same words; a warm-up or cool-down in the text must become a set; total distance is computed, never asked from the model; symptoms must be in English
Analysis	every body part the model calls a problem must appear in the swimmer's notes or symptoms; confidence is forced to *low* with less than four weeks of history

Task	What code decides or verifies
Plan	weekly volume target computed from load and analysis; no more than two hard days in a row; a session is never shorter than its main set; a caution is required when the analysis found a symptom; dates and race day set by code; days scaled proportionally if the model misses the target

Target architecture (mandatory)

Alpenstroke is deployable as **a) on-premise** and **b) air-gapped**. It is one Docker Compose project with no third-party services, so it also runs unchanged on **c) a Swiss cloud** VM; we did not test a Swiss provider.

- **On-premise** (`make run`, or `make prod` behind a reverse proxy): app + PostgreSQL. The only outbound connection is to `LLM_BASE_URL`, which can point at any OpenAI-compatible server inside the organisation's network. Fonts and scripts are served by the app itself. The public demo runs this way on a VPS behind Caddy.
- **Air-gapped** (`make model`, `make airgapped`, `make airgapped-check`): `compose.airgapped.yml` runs Apertus v1.5 8B in llama.cpp, the app and PostgreSQL on a Docker network with `internal: true`. The only container with a second network is a reverse proxy that lets the browser in on `127.0.0.1:8000`. `make airgapped-check` runs inside the app container:

```
model   http://apertus:8080/health: reachable
outside https://api.publicai.co: blocked
outside https://huggingface.co: blocked
outside https://1.1.1.1: blocked
OK: the app reaches the model and nothing else
```

Dependency	Build time	Runtime
Docker images (python:3.12-slim, postgres:16, caddy:2, llama.cpp server-b11312)	pulled once	—
Python packages (PyPI)	installed into the image	—
Model weights, 5.06 GB GGUF	`make model`, SHA-256 verified	read-only volume
Apertus endpoint	—	`LLM_BASE_URL`: local llama.cpp (air-gapped) or any OpenAI-compatible server

3. Use of Apertus

- **Models:** Apertus 70B Instruct served by Public AI (`swiss-ai/apertus-70b-instruct`, `https://api.publicai.co/v1`); `swiss-ai/Apertus-v1.5-8B` as a community Q4_K_M GGUF conversion (`Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF`, SHA-256 `a037df8d…f299`), served by llama.cpp.
- **How it is used:** inference only, three structured-output tasks (parse, analyze, plan).
- **Where it runs:** hosted endpoint for the public demo; local llama.cpp for air-gapped mode (CPU inside Docker) and for evaluation of the 8B model (Metal on an Apple M4, 16 GB).
- **Settings:** temperature 0.1 for parsing, 0.3 for analysis, 0.4 for plans, 0.0 on the retry; `max_tokens` 3000; llama.cpp with `--jinja` (the chat template from the GGUF file), `--ctx-size 12288` (longest prompt ≈2.3k tokens + answer + one retry), `--cache-ram 0`.
- **Configuration:** `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`, plus `LLM_TIMEOUT` for slow local models.
- **Prompts** live next to the code that uses them: `PARSE_SYSTEM` (`parser.py`), `ANALYZE_SYSTEM` (`analyzer.py`), `PLAN_SYSTEM` (`planner.py`). The stroke glossary and warm-up words in the parse prompt are generated from the same Python dictionaries the code checks use.
- **Other models:** none. No other LLM was used, including as a judge; all evaluation is rule-based.

4. Data

- **Parse cases** (`data/eval/parse_cases.jsonl`): 27 workout logs written by the author in English (8), Russian (7), German (4), French (4) and Italian (4), each with the expected pool, duration, effort, sets with intervals and rest, total distance and symptoms. Three splits: **dev** (16, used while changing prompts and code), **holdout** (5, written before the first prompt change; their errors were seen after

the second round), **test** (6, written before the code checks and not inspected before the run reported here).

- **Profile cases** (`data/eval/profile_cases.jsonl`): three synthetic swimmers generated by `scripts/seed.py` with fixed random seeds: *Steady* (six even weeks), *Spike* (last week's load jumped, ACWR 1.63), *Cramps* (recurring calf cramps on butterfly in the notes).
- **Model responses** (`data/eval/runs/*/responses.jsonl`): every raw answer, including retries, the output after code checks and the per-field score.
- **Licence:** all cases are original and released under CC-BY-4.0 (recorded per case). No personal data is in the repository. The tester in section 5.5 agreed to be named; his workouts are not published. `data/` is far below the 100 MB limit.

5. Evaluation

`make eval` runs every case three times (`--repeats 3`), scores each field against the expected answer and writes `responses.jsonl`, `summary.json` and `summary.md`. Two details matter for honest numbers: the Public AI gateway caches identical requests, so the evaluation adds a run tag to the system message and every repeat is a real call; and a run that fails to produce a valid answer counts as wrong, not as missing.

5.1 Parsing free text (Apertus 70B)

Setup	Cases × runs	Valid JSON	Fully correct	dev	holdout	test
v1: prompt only	21 × 3	100%	54%	71%	0%	—
v2: + code checks by quoted source, pool by pattern	27 × 3	99%	83%	83%	93%	72%
v3: + stroke accepted in any spelling, filled by code	27 × 3	**100%**	**95%**	98%	80%	**100%**

Per field in v3: pool 100%, duration 100%, effort 96%, intervals and rest 99%, sets 100%, symptoms 100%, total 100%. The step from v1 to v2 came from code, not prompt wording: prompt-only fixes for the same errors (stroke copied from the next set, pool length read as a 25 m set, rest read as a send-off) did not hold across repeats, while quoting and checking did.

5.2 Analysis and weekly plan (Apertus 70B, 3 profiles × 3 runs)

Metric	Result
Analysis: expected concern found (overload for Spike, symptom for Cramps)	100%
Analysis: no invented concern (nothing for Steady), no ungrounded body part	100%
Plan: rules hold after checks (hard days, main set, caution with symptom)	100%
Plan: goal stroke kept (reduced, not banned) for Cramps	100%
Plan: weekly volume inside target — **model alone**	67%
Plan: weekly volume inside target — after code	100%

5.3 Air-gapped model: Apertus v1.5 8B vs 70B

Metric	70B (Public AI)	8B Q4_K_M (llama.cpp, Metal)
Parse: valid answer	100%	91%
Parse: fully correct (dev / holdout / test)	95% (98 / 80 / 100)	23% (27 / 0 / 33)
Parse: pool (decided by code)	100%	100%
Parse: intervals and rest	99%	35%
Analysis: fully correct	100%	**100%**
Plan: fully correct	100%	67%
Plan: caution present when a symptom was found	100%	50%
Plan: volume inside target, model alone	67%	0%
Median seconds: parse / analysis / plan	4.8 / 3.4 / 9.2	12.1 / 15.2 / 52.1

The 8B model loses most on free-text parsing, especially intervals and rest, and is as good as 70B on analysis, where code has already computed the numbers. The first 8B run had 10 parse failures; all of them were one field (`stroke` written as "butterfly" or "IM", or missing). Accepting any spelling through the shared glossary and letting code fill a missing stroke raised valid answers from 88% to 91% and first-try answers from 43% to 63%. In air-gapped mode on CPU inside Docker (Apple M4, 11 GB for Docker) an analysis took about one minute.

5.4 What the numbers mean in the product

Parsing errors never reach the metrics silently: the parsed workout is shown as a draft and saved only after the swimmer confirms or edits it. Analysis and plans read numbers computed by code. This is why the air-gapped 8B setup is usable despite weaker parsing, and why a stronger local model (70B on a GPU server) would be a resource decision, not an architectural one.

5.5 User testing

A teammate from the Wheeling University swim team used the live demo with his own workouts and a short English instruction sheet, and the author ran one more test with synthetic workouts. We report what we found as is; nothing below was fixed before submission.

Emanuele Lugo (Italian; logged three workouts, one in Italian and two in English; named with his consent).

- *Parsing:* the Italian and the English logs were read correctly, except for one pattern: a set with two strokes (for example butterfly and backstroke in one set) was saved with only the first stroke. The cause is the data model, not the language: a set has one `stroke` field, and the code check takes the stroke from the quoted words only when exactly one stroke is named there. With two, it keeps the model's choice, which is the first one. The draft can be corrected by hand before saving.
- *Analysis:* rated as useful and accurate.
- *Plan:* too many rest days and sessions that were too short — about 3,000 m against his usual 4,000 m. The cause is the short-history path: with less than four weeks of history there is no weekly volume target (section 2), so the model received no number to aim for and played it safe. This is the honest-confidence rule working as designed, with a cost: the plan underestimates a strong swimmer who is new to the app.
- *Interface:* all three workouts were saved with the same date, because the date field defaults to today and is easy to miss. The history and the load metrics were wrong until the dates were fixed.

Author's run (three synthetic workouts written for the test, one in French, 3,500–3,700 yards each, entered on the live demo; not a tester's own data).

- *Parsing:* strokes in French (crawl, 4 nages, dos, brasse), "souple" as a cool-down, send-off versus rest ("départ 1:45" versus "15 secondes de récup", "on 1:50" versus "10 sec rest") and the French symptom, translated to English, were all read correctly. Three errors, none caught by the code checks:
  - a dropped multiplier, twice: "3 rounds of 4x100 + 100 easy" was counted once (2,500 instead of 3,500 yards), and "4x200" became 1×200 (3,000 instead of 3,600). The check verifies reps only when the model gives more than one, and every number it did use appears in the quoted words;
  - a wrong send-off: "départ 1:50" became 60 s. Send-off values are not compared with the text. They do not enter the load metrics;
  - durations written as "1h30" and "1h35" were not read; minutes were filled in by hand.
- *Analysis:* both symptoms (left knee on breaststroke kick, right calf cramp on backstroke) were found and grounded, and confidence was low as required. But with no earlier history the model read the start of the log as a spike ("from zero to 2005", the sum of three session loads, without units) and raised an overload concern, with both symptoms folded into it instead of a separate symptom concern.
- *Plan:* one rest day and 13.4 km in 2.0–2.5 km days, a recovery week that follows the overload concern. Because the analysis had no symptom concern, the code did not require cautions and the plan has none, although its summary mentions the knee and calf. Two main sets combine a send-off and a rest that cannot both hold ("10x200 freestyle @2:00 with 30s rest"), and @2:00 per 200 is far faster than the 2:25 the swimmer reported. A breaststroke kick-technique day follows knee soreness on breaststroke kick.

Together the two runs point to the same place: the model is reliable where code has fixed the numbers and the rule, and the misses are in what code does not yet check — multipliers, send-off values, the

meaning of an empty history, and how realistic a set is for this swimmer.

6. Limitations

- **Small evaluation set.** 27 parse cases and 3 profiles. One case is 6–20 percentage points of a split. Between the v2 and v3 runs, a day apart, holdout moved from 93% to 80% and test from 72% to 100%; part of that is the code change and part is run-to-run variance, which we did not separate. The holdout split was inspected after its first errors, so only the test split is blind, and it has 6 cases.
- **Synthetic profiles.** Analysis and plans are evaluated on generated histories, not real seasons.
- **ACWR.** Thresholds (0.8 / 1.3 / 1.5) come from team-sport research and are debated; they are used as a conversation starter for the swimmer, not as an injury predictor.
- **Plan scaling** changes daily distances but not the text of the main set.
- **New swimmers get cautious plans.** Without four weeks of history there is no volume target, and plans come out shorter than the swimmer's usual sessions (section 5.5).
- **One stroke per set.** Mixed sets such as "4x100 fly/back" keep only one stroke.
- **Unchecked numbers in parsing.** A dropped multiplier ("4x200" → 1×200, nested rounds) and the send-off value pass the code checks; durations like "1h30" are not read.
- **Empty history looks like a spike to the model.** With less than four weeks of data the analysis can still raise an overload concern, and symptoms inside it do not trigger the plan's caution check.
- **Set realism.** Send-offs in the plan are not compared with the swimmer's own times.
- **Grounding** is a keyword check over a list of body parts; it cannot judge advice quality.
- **The 8B model** is a community GGUF conversion; we did not test the official weights in vLLM.
- **No accounts.** The demo has no authentication; anyone with the link sees all swimmers.
- **Not medical advice.** Symptoms are flagged with a recommendation to see a coach or a professional.

7. Reproducibility

- **Run:** `cd track_2b && cp .env.example .env` (set `LLM_API_KEY`), then `make run` → http://localhost:8000 with three demo swimmers created on an empty database.
- **Tests:** `make test` — 109 unit tests, no network or database.
- **Evaluation:** `make eval` (about 20–30 minutes with 70B). For the 8B model: `make model`, then `llama-server -m models/apertus-v1.5-8b-text-q4_k_m.gguf --alias apertus-v1.5-8b --port 8080 -c 12288 --jinja -ngl 99` and `LLM_BASE_URL=http://host.docker.internal:8080/v1 LLM_NAME=apertus-v1.5-8b LLM_API_KEY=local LLM_TIMEOUT=600 make eval`.
- **Air-gapped:** `make model && make airgapped && make airgapped-check`; at least 10 GB of memory for Docker.
- **Seeds:** demo and profile histories use fixed seeds 1, 2, 3. Model sampling is not seeded; this is why every case runs three times.
- **Runs reported:** `data/eval/runs/` — 70B parse `20261007-175942`, 70B analysis and plan `20261006-204236`, 8B parse `20261007-175934`, 8B analysis and plan `20261007-171428`, v1 `20261006-202652`. Each `summary.md` records the model, endpoint and a fingerprint of every prompt.
- **Hardware:** Apple M4 MacBook Pro, 16 GB; demo server Contabo VPS, Ubuntu 24.04, Docker Compose 5.6.
- **Commit:** the last commit on `main` before the submission deadline (16 October 2026, 12:00 CEST).

8. Next steps

- Accounts and per-team visibility, so a coach sees the squad and a swimmer sees only their own data.
- A larger, real evaluation set from consenting club swimmers, with a second annotator.
- Edit the main-set text when the plan is scaled, not only the distance.
- Without a weekly target, size each day from the median of the swimmer's recent sessions (computed by code, like the target) and allow a third rest day only in a taper or after an overload or symptom concern.
- Allow several strokes per set, and make the workout date a required, visible choice.
- Parsing checks for multipliers: when the quoted words contain "N×D", reps must equal N; nested rounds as their own structure; durations like "1h30" read by code.

- With `insufficient_history`, forbid an overload concern and require a separate symptom concern whenever a workout lists symptoms.
- Check plan send-offs against the swimmer's recent times, and forbid a send-off and a rest in one set.
- Fine-tune Apertus v1.5 8B on parsing examples to close the gap with 70B for air-gapped clubs.
- Import from swim watches (FIT files) next to free text.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. The code is under the Apache License 2.0.

References

- Foster C. et al. (2001). A new approach to monitoring exercise training. *Journal of Strength and Conditioning Research*, 15(1), 109–115.
- Gabbett T. J. (2016). The training–injury prevention paradox. *British Journal of Sports Medicine*, 50(5), 273–280.
- Impellizzeri F. M. et al. (2020). Acute:chronic workload ratio: conceptual issues and fundamental pitfalls. *International Journal of Sports Physiology and Performance*, 15(6), 907–913.
- Swiss AI Initiative. Apertus models — https://huggingface.co/swiss-ai
- llama.cpp — https://github.com/ggml-org/llama.cpp